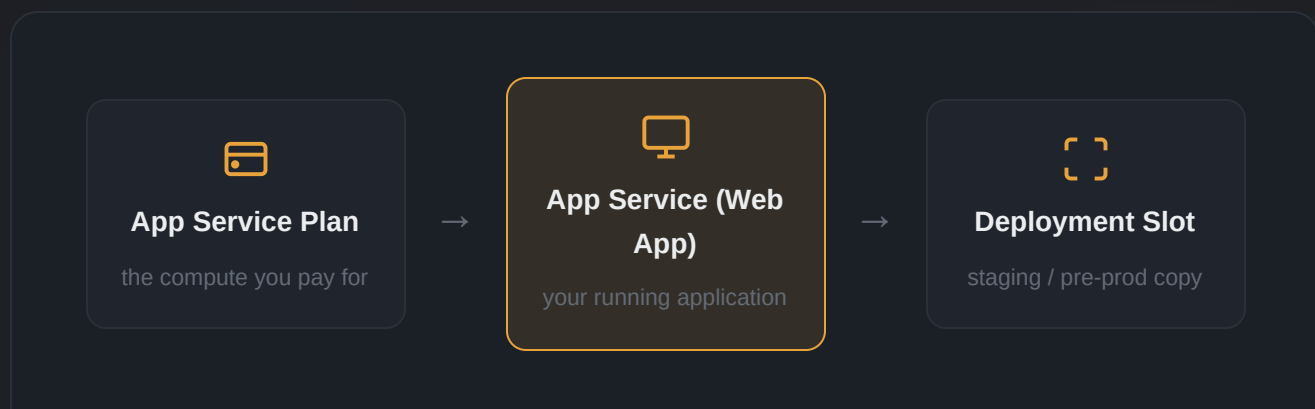


Create & Configure **Azure App Service**

App Service Plans, deployment slots, scaling, custom domains, TLS certificates, identity, networking integration, and deployment methods — everything needed to run and configure a web app in Azure.

The App Service hierarchy

Three objects, one relationship to memorize: the Plan is the compute (you pay for it); the App runs on the Plan (you can run many apps on one Plan); Deployment Slots are staged copies of the App on the same Plan.



Key rule: Multiple apps share the same Plan's CPU and memory — don't pack too many high-traffic apps onto one Plan. Slots share the Plan too.

App Service Plan tiers

FREE / SHARED

F1 / D1

No SLA, shared compute, no custom domain (F1), no scaling.

BASIC

B1–B3

Dedicated compute, custom domain + TLS, manual scale only, no slots.

STANDARD

S1–S3

Auto-scale, up to 5 deployment slots, custom domains + TLS, daily backups.

PREMIUM

P1v3–P3v3

Up to 20 slots, VNet integration, private endpoints, zone redundancy.

ISOLATED

I1v2–I3v2

Runs in your own App Service Environment (ASE) inside a VNet — highest isolation and scale.

Eight terms you need cold

Deployment Slot

A live environment running a separate version of the app (e.g. staging). Slots have their own hostnames, settings, and code — swap them into production with zero downtime.

Tip: Standard tier and above required for slots. Swapping also swaps connection strings and app settings unless they're marked "slot setting."

Slot Swap

Instantly exchanges the production slot and a staging slot — the staged version becomes production with zero downtime. Azure warms up the new slot before completing the swap.

Tip: If the swap causes issues, swap back immediately — the previous version is still sitting in the staging slot.

Scale Out (Horizontal)

Adds more instances of the app (more workers). Can be manual or automatic via autoscale rules based on CPU, memory, HTTP queue length, or a schedule.

Tip: Autoscale requires Standard tier or above. Scale out is the primary way to handle traffic spikes — not scale up.

Scale Up (Vertical)

Moves the App Service Plan to a higher tier (more CPU/RAM per instance). Requires a restart. Used when a single instance needs more resources.

Tip: Scale up to unlock features (slots, VNet integration) as much as for performance.

Custom Domain

Mapping your own domain (e.g. `app.contoso.com`) to the App Service. Requires a CNAME or A record, plus domain ownership verification via a TXT record.

Tip: Basic tier and above required for custom domains. Free tier only serves from `*.azurewebsites.net`.

Managed Certificate

A free TLS certificate issued and auto-renewed by Azure for custom domains — covers the `www` subdomain and standard subdomains.

Tip: Wildcard certs and apex domains require you to bring your own certificate. Managed certs cover standard subdomains only.

VNet Integration

Allows outbound traffic from the App Service to reach resources inside a VNet — the app initiates connections into the VNet, but VNet resources don't reach in by default.

Tip: VNet Integration is outbound only. For inbound private access, use a Private Endpoint instead.

Managed Identity

Assigns a system- or user-assigned Azure AD identity to the App Service, allowing it to authenticate to other Azure services (Key Vault, Storage, SQL) without storing credentials in config.

Tip: Always use Managed Identity over connection strings with embedded keys — no secrets to rotate or leak.

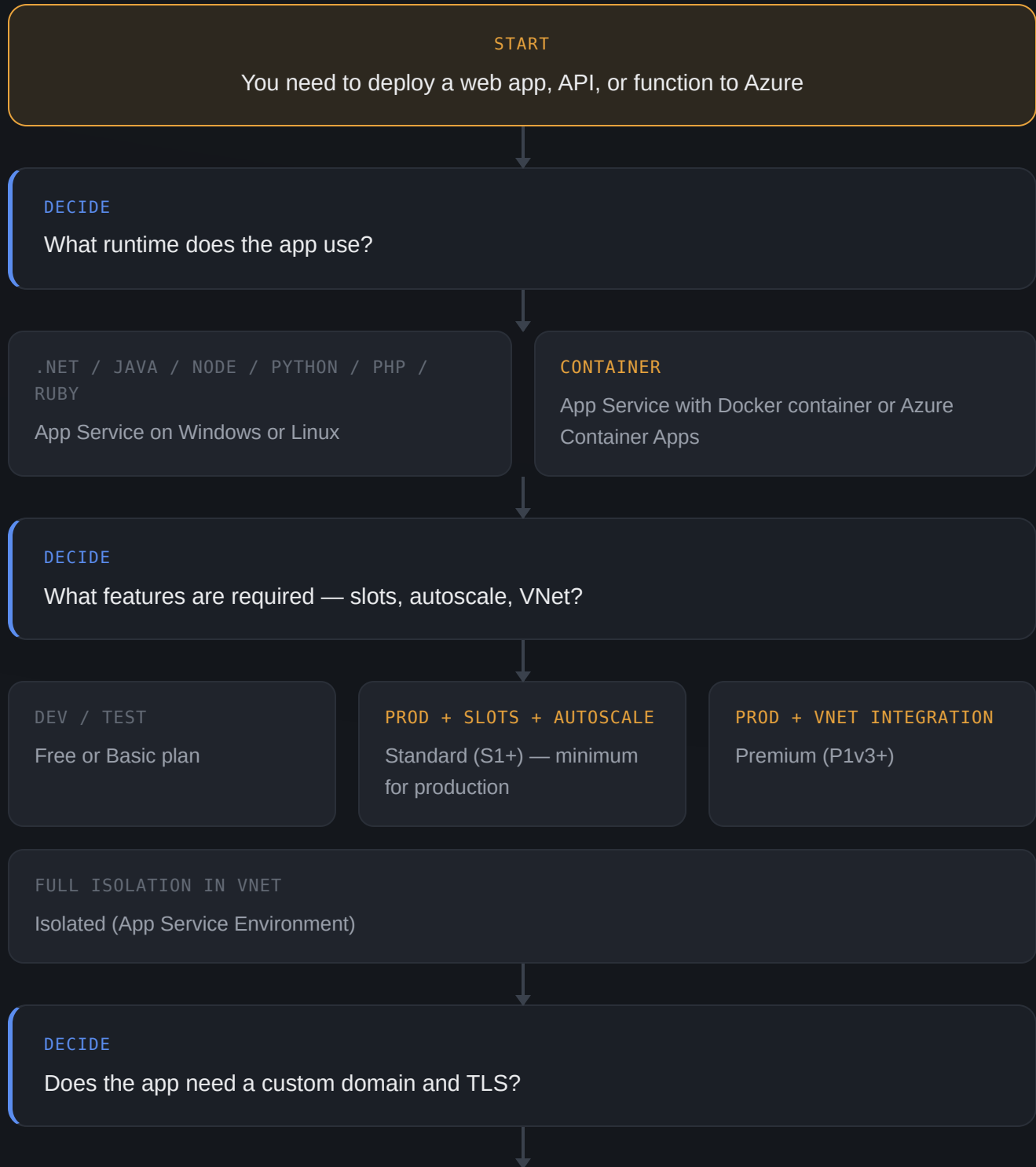
FEATURE	FREE	BASIC	STANDARD	PREMIUM	ISOLATED
Custom domain	No	Yes	Yes	Yes	Yes
TLS/SSL	No	Yes	Yes	Yes	Yes
Autoscale	No	No	Yes (5)	Yes (20)	Yes
Deployment slots	No	No	Yes (5)	Yes (20)	Yes
VNet Integration	No	No	No	Yes	Yes (ASE)

EXAM GOTCHA — SLOT SETTINGS

When you perform a slot swap, **all app settings and connection strings swap with the code by default**. To keep a setting tied to a specific slot (e.g. a staging database connection string that should NOT go to production), mark it as a **slot setting** — it stays in the slot it belongs to, regardless of swaps.

Creating and configuring an App Service

Walk through this for every new web app deployment or configuration change.



YES — SUBDOMAIN

Add custom domain → Azure-managed free cert (auto-renewed)

YES — WILDCARD / APEX

Upload your own cert or use App Service Certificate

NO

Use default

***.azurewebsites.net**

DECIDE

How will the app authenticate to other Azure services?

RECOMMENDED

Enable System-Assigned Managed Identity → grant RBAC role on target resource

LEGACY

Connection string in app settings — avoid if possible

DECIDE

How will code be deployed?

CI/CD

GitHub Actions or Azure DevOps pipeline → deploy to staging slot → swap to production

MANUAL

ZIP deploy, FTP, or VS Code App Service extension

DONE

App running with the right plan, domain, identity, and deployment pipeline

Tools for App Service

>_ Azure CLI

BEST FOR: create, configure, deploy, swap slots

Full lifecycle management in one-liners — create a plan, create the app, configure settings, and swap slots from the same script.

```
az appservice plan create \
  -g rg-prod -n plan-prod \
  --sku S1 --is-linux

az webapp create \
  -g rg-prod -n myapp-prod \
  --plan plan-prod \
  --runtime "NODE:20-lts"

az webapp deployment slot create \
  -g rg-prod -n myapp-prod \
  --slot staging

az webapp deployment slot swap \
  -g rg-prod -n myapp-prod \
  --slot staging --target-slot
production
```

>_ Azure CLI — settings & identity

BEST FOR: app settings, managed identity, scaling

Configure app settings, enable managed identity, and set up autoscale rules from the command line.

```
az webapp config appsettings set \
  -g rg-prod -n myapp-prod \
  --settings ENV=production
DB_HOST=sqlserver.db

az webapp identity assign \
  -g rg-prod -n myapp-prod

az monitor autoscale create \
  -g rg-prod --resource myapp-prod
\
  --resource-type
Microsoft.Web/serverfarms \
  --min-count 2 --max-count 10 --
count 2
```

⟨⟩ Bicep / ARM

BEST FOR: IaC — apps as code

Deploy App Service Plan, Web App, and slot configuration together — everything version-controlled and repeatable.

```
resource plan
'Microsoft.Web/serverfarms@2022-09-01' = {
  name: 'plan-prod'
  location: location
  sku: { name: 'S1', tier:
'Standard' }
  kind: 'linux'
  properties: { reserved: true }
}

resource app
'Microsoft.Web/sites@2022-09-01' =
{
  name: 'myapp-prod'
  location: location
  identity: { type:
'SystemAssigned' }
  properties: {
    serverFarmId: plan.id
    siteConfig: { linuxFxVersion:
'NODE|20-lts' }
  }
}
```

⬆ ZIP Deploy

BEST FOR: quick manual deploys, CI/CD pipelines

Package your app as a ZIP and push it directly to the Kudu endpoint — no build agent needed, works from CLI or a pipeline step.

```
az webapp deploy \
  -g rg-prod -n myapp-prod \
  --src-path ./app.zip \
  --type zip \
  --slot staging
```


Portal — Deployment Center

BEST FOR: connecting GitHub / Azure Repos CI/CD

The Deployment Center blade connects App Service directly to a GitHub repo, Azure Repos, or Bitbucket and auto-generates the GitHub Actions workflow file.

App Service → Deployment Center
→ Source: GitHub / Azure Repos
→ Branch: main
→ Save → auto-generates workflow

DEPLOY METHOD	ZERO DOWNTIME?	BEST FOR
Slot swap	Yes	Production releases — warm up staging then swap
ZIP deploy	Brief restart	Fast pushes to non-production slots
GitHub Actions / DevOps	Yes (with slot)	Automated CI/CD pipelines
FTP / FTPS	No	Legacy — avoid in production

Principles worth internalizing

VNET INTEGRATION VS PRIVATE ENDPOINT

VNet Integration = outbound traffic *from* the app into a VNet (e.g. the app calls a private SQL server). The app still has a public inbound endpoint. **Private Endpoint** = inbound traffic *to* the app arrives via a private IP inside the VNet — the app's public endpoint can be disabled entirely. You often need both: Private Endpoint for inbound isolation, VNet Integration for outbound to private resources.

EXAM GOTCHA – SLOT SETTINGS

App settings swap with the code during a slot swap **unless** they are marked as **slot settings**. Classic failure pattern: staging slot connects to a staging database via a connection string, the swap fires, and production suddenly points at the staging database because the connection string wasn't marked as a slot setting. Always mark environment-specific settings as slot settings.

Six mistakes that show up on the exam

Expecting deployment slots on Basic tier

Slots require Standard tier or above — Basic gives you dedicated compute and custom domains, but no slots and no autoscale.

Not marking environment-specific settings as slot settings

Connection strings and app settings swap with code unless explicitly marked as slot settings. Staging DB credentials will reach production if not locked to the slot.

Thinking VNet Integration gives inbound isolation

VNet Integration is outbound only — the app reaches into the VNet, but the app's public endpoint is still reachable. Use a Private Endpoint for inbound isolation.

Using a free managed cert for an apex / wildcard domain

Azure-managed free certificates only cover standard subdomains (e.g. `www`). Apex domains (`contoso.com`) and wildcards (`*.contoso.com`) require a paid or bring-your-own certificate.

Scaling up instead of scaling out for traffic

Scale up moves to a bigger instance (more CPU/RAM) — useful for feature unlocks. Scale out adds more instances — the right answer for handling increased traffic load.

Storing secrets in app settings as plain strings

App settings are visible in the portal and deployment history. Reference Key Vault secrets directly using a Key Vault reference syntax:

```
@Microsoft.KeyVault(SecretUri=...) .
```